

KB: A Facts-First Codebase Knowledge System with Multi-Objective Exploration Loss Evaluation

Jacob Rosenzweig
Independent Research
rosenjcb@gmail.com

June 2026

Abstract

LLM-based developer agents suffer from two compounding inefficiencies: each session begins with an empty context window, forcing costly re-exploration of the codebase, and current evaluation frameworks measure only whether the final answer is correct, hiding the exploration cost that produced it. We present two complementary contributions.

KB is a local-first, facts-first codebase knowledge system that converts knowledge maintenance into a side-effect of normal development work. KB indexes source code deterministically via AST analysis (tree-sitter) and documentation via sentence-level fact extraction, storing atomic facts in a SQLite graph with typed edges. At query time, a multi-pond BFS loop merges lexical (BM25/FTS5), semantic (embedding), and graph-walk evidence streams into a ranked fact pool, exiting on a scored sufficiency condition.

MOEL (Multi-Objective Exploration Loss) is a quantitative evaluation framework that measures agent efficiency under three controlled conditions: Condition N (raw filesystem baseline), Condition K (KB-equipped), and Condition O (oracle context injection). MOEL combines three normalized loss terms—LLM jury semantic loss (L_{jury}), trajectory inefficiency ($L_{\text{trajectory}}$), and resource consumption (L_{resource})—into a single weighted scalar L_{MOEL} . Bias mitigation (position debiasing, verbosity normalization, self-enhancement down-weighting, family diversity enforcement) and statistical calibration via per-judge logistic regression are built in.

The headline metric is a budget-normalized success score $S = 0.60 Q + 0.30 E_{\text{tok}} + 0.10 E_{\text{speed}}$ that blends answer quality with token economy and latency, scored identically for KB-equipped and control runs. The project grade is $\Delta S = S_K - S_N$ from a single eval artifact (Eq. 8). On the **kb** suite, a paired harvest (run 1747) yields $\Delta S = -0.058$ (behind control on quality, ahead on speed); one-shot synthesis

later cuts kb tokens to $\sim 135\text{ k}$ at $S_K = 0.84$ pending a fresh paired rerun.

1 Introduction

Large language models (LLMs) have dramatically changed how developers interact with codebases, enabling agents to autonomously resolve real-world software engineering issues [11]. Yet the dominant interaction model remains *ephemeral*: each agent session begins with an empty context window, forcing the model to re-discover facts about the codebase from scratch. This creates a compounding inefficiency—teams that rely on LLM assistants must accept that every question triggers a fresh, expensive, and error-prone tour of the filesystem.

Two failure modes characterize this status quo. First, **exploration waste**: agents read far more source files than necessary before reaching an answer, because there is no prior accumulation of structured knowledge to query. On representative SWE-Bench tasks, text-based agents spend up to 21 steps navigating via `grep` and line-range reads to locate a single target function—work that structured pre-indexed facts resolve in two or three queries [5]. Second, **context dilution**: irrelevant context degrades LLM reasoning [8], and brittle string-matching edits fail when whitespace or formatting drifts [5].

Existing retrieval-augmented generation (RAG) systems [6] address the first problem partially, but are designed for static document corpora, not living codebases with interleaved code, documentation, and decision history. Code-search benchmarks such as CodeSearchNet [4] evaluate individual retrieval steps in isolation, ignoring the sequential cost of multi-step agent exploration.

We present **KB**, a local-first, *facts-first* codebase knowledge system that:

- (1) indexes a repository’s code and documenta-

tion into a persistent SQLite store of atomic, citable facts via deterministic AST analysis and sentence-level fact extraction;

- (2) exposes those facts to LLM agents through a multi-pond BFS retrieval loop that merges lexical (BM25), semantic (embedding), and graph-walk evidence streams without manual query engineering.

Critically, KB’s value is only as compelling as the evidence that it actually reduces agent exploration cost. Rubric-based Q&A scoring—the standard in prior agent evaluation—captures answer quality but is blind to *how* the agent arrived at that answer. A system that reads 40 files before synthesizing a correct response and one that queries two facts look identical under a pass/fail quality signal.

To fill this gap, we introduce the **Multi-Objective Exploration Loss (MOEL)** evaluation framework. The harvest runner scores both KB (K) and a real-agent control (N) on the same eight questions and reports the headline grade $\Delta S = S_K - S_N$ (Eq. 8). A complementary deep pipeline runs tasks under three conditions (N, K, O) and aggregates exploration loss into L_{MOEL} .

Figure 1 illustrates the end-to-end system. Our contributions are:

1. **KB**: a facts-first codebase knowledge system with deterministic AST indexing and a multi-pond hybrid retrieval orchestrator.
2. **MOEL**: a multi-objective evaluation framework comprising three normalized loss functions (L_{jury} , $L_{\text{trajectory}}$, L_{resource}), bias-mitigated LLM jury scoring, and logistic-regression calibration on a 20-sample human-annotated set.
3. **Empirical results**: harvest evaluation compares condition K (**kb query**, one-shot synthesis) to condition N (real coding agent, no KB) via $\Delta S = S_K - S_N$ in a single artifact. On the KB self-check suite, $S_K = 0.84$ at ~ 135 k tokens for eight queries (June 2026).

2 Related Work

2.1 LLM-Based Code Agents and Tools

SWE-Agent [11] navigates repositories through iterative file-read and string-edit operations. Agentless [10] takes a non-agentic approach—a

[FIGURE 1 — System Overview]

A three-panel horizontal flow diagram:

Left panel — Developer working directory with source files, markdown, and git history feeding into **kb init** (AST indexer + sentence segmenter + fact extractor).

Center panel — SQLite knowledge store: **facts** table (doc + code facts), **fact_edges** graph, **fact_embeddings** semantic index, **documents** (human-readable artifacts).

Right panel — LLM agent loop: **kb query** dispatches multi-pond BFS retrieval, passes ranked facts to LLM, returns grounded prose answer.

Below all panels: MOEL harness comparing Condition N (raw filesystem), K (kb-enabled), O (oracle) with L_{MOEL} scalar output.

Figure 1: End-to-end KB system and MOEL evaluation harness.

fixed localization-then-repair pipeline that deliberately avoids giving the LLM autonomous tool-use—yet still requires direct file reads without pre-indexed context. AutoCodeRover [12] augments localization with AST-based code search APIs (e.g., `search_method_in_file`) that retrieve class and method implementations from a structured AST index. CODESTRUCT [5] replaces text-based edits with AST-grounded primitives (`readCode/editCode`), reducing input tokens by 12–38% and improving SWE-Bench Verified Pass@1 by up to 5.0pp. KB is complementary to these systems: whereas CODESTRUCT improves the *action interface* once a target is located, KB improves *context retrieval* by replacing filesystem exploration with structured fact lookup. The two approaches can be composed.

2.2 Retrieval-Augmented Generation

RAG [6] grounds LLM answers in retrieved passages, but standard RAG pipelines are designed for static document collections. They do not model the inter-entity relationships that characterize codebases (function calls, class hierarchies, module imports), and they treat retrieval as a one-shot lookup rather than an adaptive loop. KB’s multi-pond BFS loop merges five evidence streams per pass and exits on a scored sufficiency condition, making retrieval quality a function of graph density rather than fixed index size.

2.3 LLM Evaluation Frameworks

Evaluating LLM outputs with another LLM (LLM-as-judge) is established in MT-Bench [13], which identifies position bias, verbosity bias, and self-enhancement bias as the primary failure modes. AlpacaFarm [2] applies a related idea—using LLMs to simulate human preference feedback—for RLHF training at lower cost. PandaLM [9] trains a dedicated judge model for reliable instruction-tuning evaluation. MOEL’s jury module addresses the same biases at inference time through position debiasing (swapped-order consistency), verbosity normalization ($\log_{1+}$ length scaling), self-enhancement down-weighting ($\times 0.5$ for same-family judges), and family diversity enforcement—without requiring a fine-tuned judge. Unlike prior work, MOEL is not outcome-only: it combines the LLM jury with an AST-structural loss and trajectory/resource losses so that efficiency failures are numerically visible.

2.4 Agent Efficiency Measurement

SWE-ContextBench measures token cost and cache efficiency across context injection strategies; CodeScaleBench evaluates tool validity across repository scale classes. Neither framework provides a unified scalar loss that spans correctness, trajectory, and resource consumption simultaneously. The closest prior work is SWE-Atlas, which verifies agent-declared file manifests against live `git diff` output. MOEL’s `ManifestValidator` adopts the same live-diff approach and extends it with a `MutationValidator` that applies tree-sitter AST body stubs to confirm causal linkage between agent changes and test coverage.

2.5 AST-Based Code Analysis

Tree-sitter and GumTree [3] provide fine-grained AST differencing and incremental parsing. `code2vec` [1] uses AST path embeddings for code representation learning. MOEL’s L_{AST} adopts a simpler but interpretable measure: Jaccard distance over named exported declarations, computed deterministically without learned components, making it reproducible across model versions.

3 KB: A Facts-First Codebase Knowledge System

3.1 Knowledge Representation

KB represents a codebase as a set of atomic, citable *facts* stored in a SQLite database. Each fact is a natural-language sentence paired with:

- a `source_kind` (`import_code` for AST-derived facts, `import_doc` for sentence-segmented markdown);
- a `source_ref` (e.g., `code:src/tools/indexer.ts@parseAST`);
- optional embedding vector for semantic search;
- a soft-deletion tombstone for incremental invalidation.

Facts are connected by typed edges in a `fact_edges` table, forming a lightweight semantic graph. This graph enables breadth-first neighborhood expansion during retrieval—a capability not available in flat vector indices.

3.2 Indexing Pipeline

`kb init` executes a fully deterministic, two-phase indexing pipeline. No LLM is invoked and no API cost is incurred.

Phase 1 — AST Code Indexing. For each source file, KB invokes the `web-tree-sitter` WASM runtime with per-language export queries to extract named declarations (functions, classes, interfaces, type aliases, constants, module items). Each symbol becomes a code fact with its qualified source reference. Import edges between symbols are recorded in `fact_edges`, forming the structural backbone of the graph.

Phase 2 — Document Fact Extraction. Human-readable sources (README files, markdown documentation) are segmented into sentences via deterministic NLP segmentation. Each sentence becomes a document fact stored verbatim in the `facts` table. Embeddings are computed lazily at query time, not during init.

The entire init pipeline for the raylib C library (373–698 entities across runs) completes in under 3 minutes on a laptop with no API cost.

[FIGURE 2 — KB Indexing and Retrieval Architecture]

A two-column diagram.

Left column (Init): Source files → tree-sitter AST parser → named declarations → **facts** (code). Markdown/README → sentence segmenter → **facts** (doc). Both feed into **fact_edges** (graph) and **fact_embeddings** (semantic).

Right column (Query): Query string → intent router → multi-pond BFS loop (labeled with the five evidence streams: edge neighbors, primary FTS, pond FTS, concept frontier, concept rows) → scored fact pool → sufficiency check (≥ 10 facts ≥ 0.40) → LLM synthesis → grounded prose answer with evidence header.

Arrow between columns labeled “graph-augmented query expansion”.

Figure 2: KB indexing (left) and multi-pond BFS retrieval (right).

3.3 Retrieval: Multi-Pond BFS

At query time, KB executes a *deep* retrieval loop that maintains a set of *exploration ponds*—independent BFS frontiers seeded from different sub-queries—so that one dominant code neighborhood cannot monopolize the evidence budget.

Each iteration of the loop merges five candidate streams:

1. **Edge neighbors:** one BFS hop from the active pond’s frontier via **fact_edges**.
2. **Primary lexical:** BM25 [7] FTS5 search for the original query.
3. **Pond lexical:** FTS5 search for the active pond’s sub-query.
4. **Concept frontier:** facts sharing any concept token in the active concept set.
5. **Concept rows:** broader concept-union expansion.

Facts from the first pass are promoted as *anchors* and receive a +0.10 score boost in later passes, ensuring stable grounding. The loop exits when any of the following conditions is satisfied: (a) ≥ 10 facts score ≥ 0.40 against the query (*answerable plateau*), (b) all ponds are exhausted, or (c) the fact budget (500 facts) is reached.

Graph-augmented query expansion may optionally rewrite or widen the query string before the loop begins, based on entity relationships in the stored graph. The *pre-expansion* user question is preserved for answer synthesis so graph symbol names do not pollute scaffold or prompt text.

3.4 Answer synthesis

Retrieval output is identical for **kb query** and **chat**; synthesis paths differ:

- **One-shot** (kb query) — `enrichReadDocumentsAnswerWithLLM()` issues a single LLM call over at most 150 facts (2000 characters each). Eval harvest and programmatic agents use this path.
- **Multi-turn (chat)** — `runChatSynthesis()` may invoke a `query_kb` tool across several rounds for targeted follow-up retrieval before producing the final answer.

Both paths truncate fact text and share the same relevance filter when result counts exceed 20 facts.

4 MOEL: Multi-Objective Exploration Loss

4.1 Motivation

Standard evaluation of LLM agents on code tasks measures whether the final answer is correct. This binary signal hides the cost of exploration: an agent that issues 50 tool calls to reach a correct answer and one that issues 3 are indistinguishable under pass/fail scoring. Similarly, LLM-as-judge evaluators have known failure modes (position bias, verbosity inflation, self-enhancement) that inflate quality estimates when judges are not calibrated [13].

MOEL addresses both problems by combining three normalized loss components into a single scalar, evaluated under three controlled conditions per task.

4.2 Three-Condition Evaluation Design

Every MOEL task runs under three conditions:

The primary hypothesis is $L_{\text{MOEL}}(N) > L_{\text{MOEL}}(K)$; secondary goal is $L_{\text{MOEL}}(K) \approx L_{\text{MOEL}}(O)$ as the knowledge base matures. `compareConditions()` checks this automatically from the per-condition result objects.

Cond.	Tools available	Purpose
N (raw FS)	<code>read_file</code> , <code>list_directory</code> , <code>search_file_contents</code>	Worst-case baseline
K (KB)	Full KB tool registry	Primary experiment
O (Oracle)	Minimal facts injected in system prompt	Ceiling

4.3 Loss Functions

L_{jury} — LLM Jury Semantic Loss

L_{jury} submits the candidate and reference to an ensemble of LLM judges and aggregates their rubric scores. Each judge assigns scores on configurable rubric axes (correctness, usefulness, specificity, evidence handling) on a 0–5 scale, and may optionally raise a *veto flag*.

Bias mitigation. Four debiasing mechanisms are applied:

1. **Verbosity normalization:** a $\log_{1+}$ length-scaling term is added to the rubric, penalizing unnecessary verbosity without penalizing accurate, detailed responses.
2. **Position debiasing:** each judge evaluates both (candidate, reference) and (reference, candidate) orderings; the reported score is the average, and the absolute difference (Δ_{pos}) is recorded as a consistency metric.
3. **Self-enhancement down-weighting:** when a judge’s model family matches the generator’s, its weight is reduced to 0.5.
4. **Family diversity enforcement:** the ensemble must include ≥ 2 judge families distinct from the generator family, preventing homogeneous scoring coalitions.

Minority-veto policy. If ≥ 2 judges veto, $L_{\text{jury}} = 1.0$; otherwise it equals $\bar{s}_w$, the weighted mean of per-judge normalized scores:

$$L_{\text{jury}} = \begin{cases} 1.0 & \text{veto count} \geq 2 \\ \bar{s}_w & \text{otherwise} \end{cases} \quad (1)$$

Statistical calibration. Jury scores are calibrated using per-judge logistic regression:

$$P(\text{correct} \mid s) = \sigma(\beta_0 + \beta_1 s) \quad (2)$$

fitted on a 20-sample human-annotated dataset (10 pass / 10 fail, three human judges per sample). Generalization error is estimated via leave-one-out cross-validation. The pipeline is invoked once per judge

model and stores (β_0, β_1) coefficients for use at scoring time.

$L_{\text{trajectory}}$ — Trajectory Inefficiency Loss

$L_{\text{trajectory}}$ penalizes two failure modes: taking more steps than a configured ceiling, and repeating identical tool calls. Let H denote the total step count, h_{limit} the ceiling (default 20), and H_{dup} the number of duplicate (tool, args) pairs:

$$L_{\text{traj}} = \min\left(\frac{1}{2} \min\left(\frac{H}{h_{\text{lim}}}, 1\right) + \frac{1}{2} \frac{H_{\text{dup}}}{H}, 1\right) \quad (3)$$

Duplicate detection normalizes argument keys (alphabetical sort, whitespace trim) so that logically identical calls with different JSON serializations are correctly collapsed.

L_{resource} — Resource Consumption Loss

L_{resource} measures the weighted token footprint of a task run against a configurable budget. Following Anthropic’s prompt caching pricing, cached tokens are discounted by $\delta = 0.1$, and output tokens are weighted at $\gamma = 1.0$:

$$L_{\text{resource}} = \min\left(\frac{C_{\text{fresh}} + \delta \cdot C_{\text{cached}} + \gamma \cdot C_{\text{output}}}{\text{budget}}, 1\right) \quad (4)$$

Token counts are drawn from per-step records that distinguish fresh, cached, and output tokens; aggregated totals that lack this split are not used.

4.4 MOEL Aggregation

The three components combine into a single scalar:

$$L_{\text{MOEL}} = w_C \cdot L_{\text{jury}} + w_T \cdot L_{\text{traj}} + w_R \cdot L_{\text{res}} \quad (5)$$

Default weights: $w_C = 0.5$, $w_T = 0.3$, $w_R = 0.2$. The constraint $w_C + w_T + w_R = 1$ is validated to within 10^{-6} and all loss terms are bounded to $[0, 1]$.

4.5 Benchmark Alignment

MOEL’s three-condition design (N/K/O) mirrors SWE-ContextBench’s context injection conditions; L_{resource} replicates that benchmark’s token-cost metric with the additional fresh/cached split. CodeScaleBench’s tool-validity tier corresponds to MOEL’s condition comparison: Condition N uses only primitive filesystem tools, Condition K adds the full KB tool registry.

5 Experiments

5.1 Evaluation Targets

Raylib. Our primary external benchmark is the [raylib C library](#)—a mature, well-documented game-programming framework. Raylib is chosen because it is not the KB codebase itself (eliminating evaluator familiarity bias), has rich cross-module structure suitable for graph-based retrieval, and has stable upstream documentation.

KB self-check. As a product smoke test, we also evaluate KB against the KB codebase itself (suite **kb**). This self-evaluation tests whether KB can answer questions about its own architecture—a strict test of retrieval grounding, since the evaluator has ground truth of the correct answers.

5.2 Protocol: kb vs control in one artifact

Each harvest run executes, in order: (1) clone and index the suite repo (**kb init** or reuse session, **kb scan**); (2) eight **kb query** calls (condition **K**) using *one-shot* synthesis (§5.6); (3) the same eight questions handed to a real coding agent with no KB (condition **N**, Claude Code headless by default); (4) LLM auto-scoring ($\times 3$ averaged) on four rubric axes for both sides.

Both sides land in a single `~/kb/evaluations/<run>/artifact.json`. The control block holds S_N ; the kb block holds S_K ; the **comparison** block holds pairwise deltas. Passing `--skip-control` omits steps (3)–(4) and produces no headline grade.

5.3 Scoring Rubric

Each suite covers eight questions spanning capabilities, architecture, installation, configuration, platform specifics, conventions, gotchas, and roadmap—topics that stress different retrieval paths through the fact graph. Answers are scored on four axes (0–4):

Axis	Criterion
Correctness	Factually grounded in the repo/KB
Usefulness	Helps a developer act or understand
Specificity	Uses concrete repo-specific details
Evidence Handling	Constrained to evidence; acknowledges uncertainty

5.4 Success Score and Headline Verdict (ΔS)

The harvest pipeline’s single scalar per side is $S \in [0, 1]$ (higher is better). It blends answer quality with operational cost so that inflating context to raise quality is penalized:

$$S = w_Q \cdot Q + w_T \cdot E_{\text{tok}} + w_V \cdot E_{\text{speed}} \quad (6)$$

with default weights $w_Q = 0.60$, $w_T = 0.30$, $w_V = 0.10$:

$$Q = \frac{\bar{c} + \bar{u}}{8}, \quad E_{\text{tok}} = 1 - \min\left(\frac{T}{B_T}, 1\right), \quad E_{\text{speed}} = 1 - \min\left(\frac{\tau}{B_\tau}, 1\right) \quad (7)$$

where $\bar{c}, \bar{u}$ are mean correctness and usefulness (0–4), T is total input+output tokens for the eight-question side, and τ is wall-clock latency. Budgets $B_T = 10^6$ tokens and $B_\tau = 6 \times 10^5$ ms are fixed (budget-absolute), so S is stable across runs. KB (K) and control (N) use the identical formula.

Headline project grade. The MOEL-aligned verdict for whether KB beats the real-agent baseline is the *difference* in the same artifact:

$$\Delta S = S_K - S_N = \text{comparison.success_score.delta_kb_minus_control} \quad (8)$$

Implemented in `scripts/eval-shared.mjs` (**kbControlVerdict**): $\Delta S \geq +0.02 \Rightarrow$ KB *ahead of control*; $\Delta S \leq -0.02 \Rightarrow$ *behind*; otherwise *on par*. Component deltas in `artifact.comparison` localize where KB wins or loses (e.g. higher Q but lower E_{speed} when the control agent explores longer).

Secondary diagnostic. Pass rate (correctness ≥ 3 and usefulness ≥ 3 on $\geq 6/8$ questions) and per-axis means remain useful for debugging; they do not replace ΔS as the headline grade.

Scoring stability. The auto-scorer (Gemini 2.5 Flash) is non-deterministic even at temperature 0; we average three scorer calls per question.

5.5 Results: kb vs control

Table 1 reports the headline comparison on the kb self-check suite (eight architecture questions, same judge, same artifact). Condition K is **kb query**; condition N is a headless coding agent (Claude Code, no KB/MCP) exploring the clone. Run `kb-2026-06-12-1747`, June 2026.

Control wins on quality (Q) and pass rate; KB wins on speed ($E_{\text{speed}} = 0.77$ vs. 0.14) and uses

Table 1: KB vs control on the kb suite ($n = 8$ questions).

Metric	K (kb)	N (ctrl)	Δ
Success S	0.754	0.812	-0.058
Pass rate	0.875	1.000	-0.125
Correctness	3.11	3.88	-0.76
Usefulness	3.75	4.00	-0.25
Tokens (8Q)	452k	307k	
Time (8Q)	139s	516s	
<i>Verdict: behind control ($\Delta S = -0.058$)</i>			

fewer tokens despite higher per-query synthesis cost in this run (multi-turn chat synthesis on the kb path). Switching kb query to one-shot synthesis (June 2026) cuts kb-side tokens to ~ 135 k at $S_K = 0.84$ (run 2351); a fresh paired control run is needed to update ΔS under the new synthesis path.

5.6 Query synthesis: one-shot vs chat

Retrieval is shared between kb query and chat; synthesis diverges:

- **kb query** — one-shot `enrichReadDocumentsAnswerWithLLM()`: a single LLM call over the capped fact list. Used by the eval harvest and programmatic agents.
- **kb chat** — multi-turn `runChatSynthesis()` with optional `query_kb` tool rounds for targeted follow-up retrieval.

This split keeps eval token costs predictable while preserving iterative exploration for interactive sessions.

6 Conclusion

We introduced KB, a local-first, facts-first codebase knowledge system. KB’s deterministic AST indexing and multi-pond BFS retrieval provide agents with structured, persistent codebase knowledge without per-session re-exploration. kb query now uses one-shot synthesis over a capped fact pool, cutting eval token cost by $\sim 70\%$ while maintaining competitive rubric scores on the KB self-check suite ($S_K = 0.84$, pass rate 0.75).

We also introduced MOEL, a multi-objective evaluation framework spanning two pipelines: (1) the harvest runner, whose headline grade is $\Delta S = S_K - S_N$ from a single artifact comparing kb query to a real coding agent on the same eight questions; and (2) the

deep MOEL harness (L_{MOEL}), which measures exploration loss under N/K/O conditions per task. Bias-mitigated jury ensemble, statistical calibration, and programmatic validators distinguish efficient exploration from expensive correctness.

Limitations. Paired ΔS requires both eval phases. Kb-only runs report S_K but cannot emit the headline verdict until the control agent runs in the same artifact.

Calibration sample size is small. The 20-sample human-annotated dataset is sufficient for per-judge logistic regression but does not support fine-grained calibration per rubric axis.

Knowledge base scope. KB currently operates on per-file ASTs. Cross-file dependencies are captured through edge relationships but are not modeled explicitly at init time.

Future Work.

- **Full MOEL N/K/O runs:** execute the `moel-kb` task library under all three conditions and report L_{MOEL} per task.
- **Raylib ΔS :** complete a paired kb+control harvest on the external raylib suite with one-shot synthesis and report ΔS .
- **Synthesis accuracy:** reduce caller-tracing and contradictory-fact errors on architecture questions without restoring unbounded multi-turn token cost.
- **Calibration expansion:** grow the human-annotated set to 100+ samples with axis-level labels.

Reproducibility. Evaluation artifacts live under `~/kb/evaluations/<run-name>/artifact.json` (schema version 2). Question packs: `eval/suites/{raylib,kb,fzf}.yaml`. Headline grade: `comparison.success_score.delta_kb_minus_control`. Implementation: `scripts/eval-run.mjs`, `scripts/eval-shared.mjs`, `scripts/control-core.mjs`, `eval/losses/`.

References

- [1] Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. code2vec: Learning distributed representations of code. *Proceedings of the ACM on Programming Languages*, 3(POPL), 2019.

- [2] Yann Dubois, Chen Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, et al. AlpacaFarm: A simulation framework for methods that learn from human feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [3] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. Fine-grained and accurate source code differencing. *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE)*, 2014.
- [4] Hamel Husain, Ho-Howard Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. CodeSearchNet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436*, 2019.
- [5] Myeongsoo Kim, Joe Hsu, Dingmin Wang, Shweta Garg, Varun Kumar, and Murali Krishna Ramanathan. CODESTRUCT: Code agents over structured action spaces. *arXiv preprint arXiv:2604.05407*, 2025.
- [6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petrov, Vladimir Karpukhin, Yinfei Yang, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [7] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 2009.
- [8] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [9] Yidong Wang, Zhuohao Yu, Zhengran Zeng, Linyi Yang, Cunxiang Wang, Hao Chen, et al. PandaLM: An automatic evaluation benchmark for LLM instruction tuning optimization. *arXiv preprint arXiv:2306.05087*, 2023.
- [10] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 1(FSE), 2024.
- [11] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [12] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024.
- [13] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.